ENG5009 Advanced Control 5: Assignment

Nathan Sidi Bakari

April 2026

1 Abstract

This report describes the development of a controller for the ENG5009 mobile robot simulator assignment. The work was split into two parts. Task 1 used a simple waypoint controller to drive the robot to target points using the supplied robot position states. Task 2 extended this into a hybrid navigation controller that combined waypoint tracking, a neural-network obstacle avoidance layer, and a wall-follow recovery mode. The final control structure used a supervisory priority: stop, wall-follow, ANN avoidance, then waypoint tracking. The resulting controller kept the robot moving in the supplied obstacle field more reliably than the individual controllers used in isolation. Its strengths were simplicity, interpretability, and recovery from deadlock. Its limitations were the narrow sensor model, threshold-based switching, and a path that was feasible rather than optimal.

2 Introduction

Mobile robot navigation is rarely just a path planning problem. Even in a small map, the controller must point the robot towards a goal, avoid nearby obstacles, and recover when local decisions stop making progress [1, 2].

The first task was to drive the robot to a selected point using the supplied simulator states and helper functions. The second task was to guide the robot through a series of waypoints while avoiding obstacles. The MATLAB material supplied the simulator framework, robot model, wall generation, sensor model, and drawing utilities, but the controllers discussed in this report were developed progressively during the lab work and assignment tasks. The task was therefore to design, implement, and tune the control logic within the provided simulation environment.

The final solution used a layered controller. A waypoint tracker provided the goal direction. A threshold neural network reacted to nearby obstacles. A wall-follow mode was added as a recovery layer when the ANN became ineffective near frontal walls or local symmetry. This followed an engineering design principle: start with the simplest controller that can be implemented and tested, then add logic only when specific failure cases justify it. This report explains how that controller was built, how it was tuned, what worked well, and where its limits still remain.

Reactive obstacle avoidance is known to struggle with deadlock, oscillation, and loss of progress in constrained cases, which motivates the use of recovery and supervisory logic [1, 3]. The contribution of this work is, thus, a low-complexity hybrid supervisory controller that addresses those limitations while remaining easy to verify, tune, and justify step by step.

3 Literature review

Obstacle avoidance is a fundamental component of mobile robot navigation. At a basic level, a robot must move through an environment while avoiding collisions with walls, obstacles, or other

agents. In practice, this problem extends beyond sensing alone and involves motion planning, control design, uncertainty handling, and recovery when local decisions fail [1, 2].

The literature distinguishes between global and local planning strategies. Global planning assumes that a map of the environment is available, allowing the robot to compute a complete path in advance. In contrast, local planning relies on real-time sensor data to react to nearby obstacles. For applications such as domestic service robots, local methods are particularly important because environments are dynamic, cluttered, and only partially known [2].

Early work in mobile robot navigation focused on minimal-information environments. Lumelsky and Stepanov introduced boundary-following strategies, commonly referred to as Bug algorithms, in which the robot moves directly toward a goal when possible and switches to obstacle boundary tracking when blocked. These methods established a key principle: direct goal-seeking behaviour alone is insufficient in unknown environments, and reactive strategies are required to guarantee progress [4].

Subsequent developments introduced more continuous and computationally efficient reactive planners. The vector field histogram (VFH) method transformed local obstacle information into candidate steering directions that could be evaluated in real time. This represented an important step in linking perception and control within a single framework [5].

Further refinements led to improved variants such as VFH+, which incorporated robot geometry and steering constraints to produce smoother and safer motion. In parallel, the dynamic window approach (DWA) introduced planning directly in velocity space, selecting admissible translational and rotational velocities that satisfy both dynamic constraints and collision avoidance requirements [6, 7].

Despite these advances, local obstacle avoidance methods exhibit well-known limitations. One of the most significant is the local minimum problem, particularly evident in artificial potential field methods, where attractive and repulsive forces can balance and trap the robot in confined geometries [8].

Another common issue is oscillatory behaviour, where the robot alternates between similar control actions without making meaningful progress. This is typically caused by local symmetry in the environment, sensor noise, or overly reactive control laws [6, 7].

To address these limitations, many systems incorporate explicit recovery mechanisms. Recovery strategies detect when the robot is no longer making progress and temporarily override the nominal controller to restore motion. Examples include reversing, reorientation, or boundary-following behaviours [3, 1].

Recent research indicates a clear shift towards hybrid navigation architectures. Classical reactive methods such as VFH and DWA remain widely used due to their computational efficiency and interpretability. However, they are increasingly combined with higher-level planning, sensor fusion, and learning-based components to improve robustness in complex environments [1, 2].

Overall, the literature demonstrates that effective obstacle avoidance is not solely a matter of local reaction. Robust performance requires a combination of goal-directed behaviour, reactive avoidance, and recovery strategies. The evolution of the field from simple reactive rules to hybrid architectures directly motivates the use of layered control approaches in practical mobile robot systems [4, 6, 1, 2, 3].

4 Methodology

4.1 Robot model and simulation environment

All simulations were run in the supplied MATLAB framework. The robot dynamics were handled by `DynamicalModel.m`, which propagates a 24-state model including motor dynamics, body motion, and heading. For control design, the most important states were forward speed

`state(13)`, yaw rate `state(18)`, position `state(19:20)`, and heading angle `state(24)`.

The simulation step size was 0.05 s and the final test duration was 120 s. The workspace was a 10 m by 10 m bounded map with five internal wall segments generated using the supplied wall-generation utility. The environment was represented as an occupancy grid for sensing and as line segments for plotting.

The plotting convention in the supplied scripts uses horizontal axis = y and vertical axis = x. This is consistent throughout the simulator and explains why the plotted trajectories appear visually transposed relative to the state ordering.

Figure 1 shows the environment used for testing, and Table 1 lists the waypoint sequence used in the final hybrid controller run.

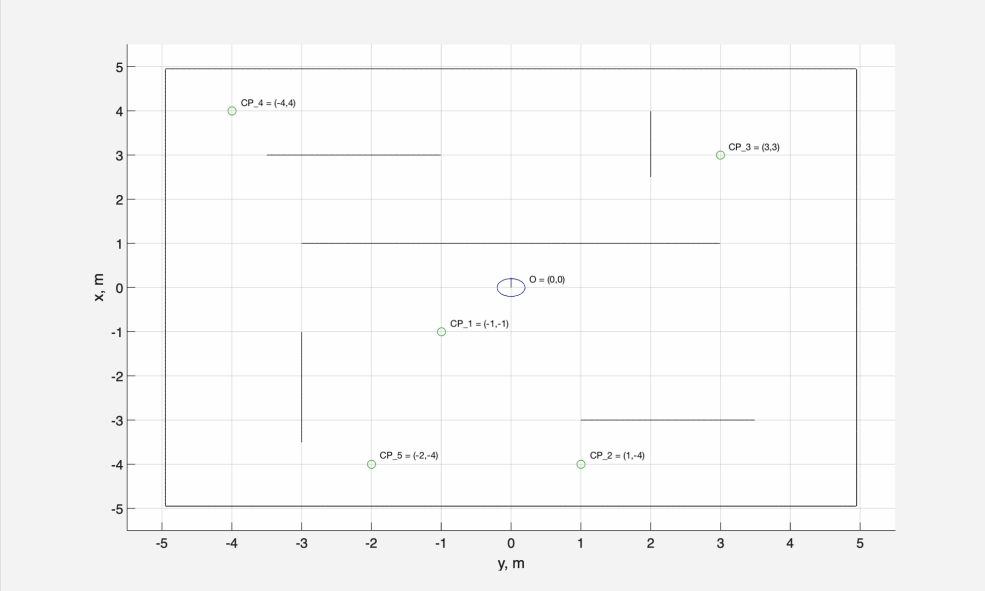

Figure 1: Environment and waypoint layout used to test whether the controller can complete the required route while repeatedly interacting with obstacles.

Table 1: Waypoint sequence used in the final hybrid controller simulation.

Waypoint	y (m)	x (m)
Start	0	0
1	-1	-1
2	1	-4
3	3	3
4	-4	4
5	-2	-4

4.2 Sensor model

The robot uses a local sensing model. `Sensor.m` returns two forward-looking distance values: a left distance and a right distance. These are generated by scanning obstacle cells inside two forward sectors from a point ahead of the robot centre. The maximum reported distance is 1 m when no obstacle is detected within the sensor range.

This matters because the controller does not have access to a map, side-wall distances, or a global obstacle representation. It only knows whether something is close on the forward-left or forward-right side. Reviews of local obstacle avoidance note that limited local sensing can

create ambiguity in frontal obstacle encounters [1, 2]. That is the main sensing limitation in the present controller.

To reduce visible wall clipping, the raw sensor values were converted into body clearances using a fixed safety margin:

$$L_S^{clear} = \max(0, L_S - 0.22), \qquad R_S^{clear} = \max(0, R_S - 0.22)$$

This makes the controller behave as if the robot body is larger than the raw sensor origin suggests.

4.3 Task 1: waypoint tracking controller

The Task 1 controller was based on the supplied helper function `ComputeHeadingAngle`. At each time step, the current waypoint was compared with the robot position to obtain the desired heading angle. The heading error was then wrapped into the range $[-\pi, \pi]$ using

$$e_\psi = \operatorname{atan2}\left(\sin(\psi_d - \psi), \cos(\psi_d - \psi)\right)$$

where ψ_d is the desired heading and ψ is the current robot heading.

The control logic was kept simple at this stage. If the robot was not aligned with the waypoint, it rotated in place. If it was aligned, it drove forward. A hysteresis variable, `isAligned`, was kept from the original bang-bang controller so the robot did not switch rapidly between rotate and drive. The controller used the simplest structure that met the task, then was refined only when the observed behaviour showed a need.

The final tuned waypoint block used a two-level turning rule. For larger heading errors, the robot used a stronger in-place turn. For smaller heading errors close to the target heading, it used a weaker turn. Once the heading error entered the alignment band, the robot drove forward with equal left and right motor voltages. This kept the original rotate-then-drive structure, but reduced the worst overshoot seen with a single fixed turn command.

Table 2 summarises the parameters used in the waypoint controller, and Figure 2 supports the later discussion of its open-space behaviour.

Table 2: Key waypoint-tracking parameters in the final script.

Parameter	Value	Purpose
Position tolerance	0.05 m	Waypoint reached condition
Alignment threshold	1°	Enter drive mode
Slow-turn threshold	10°	Switch between weak and strong turn
Misalignment threshold	10°	Leave drive mode
Weak turn voltage	1 V	Fine heading correction
Strong turn voltage	2 V	Coarse heading correction
Forward voltage	5 V	Straight-line motion

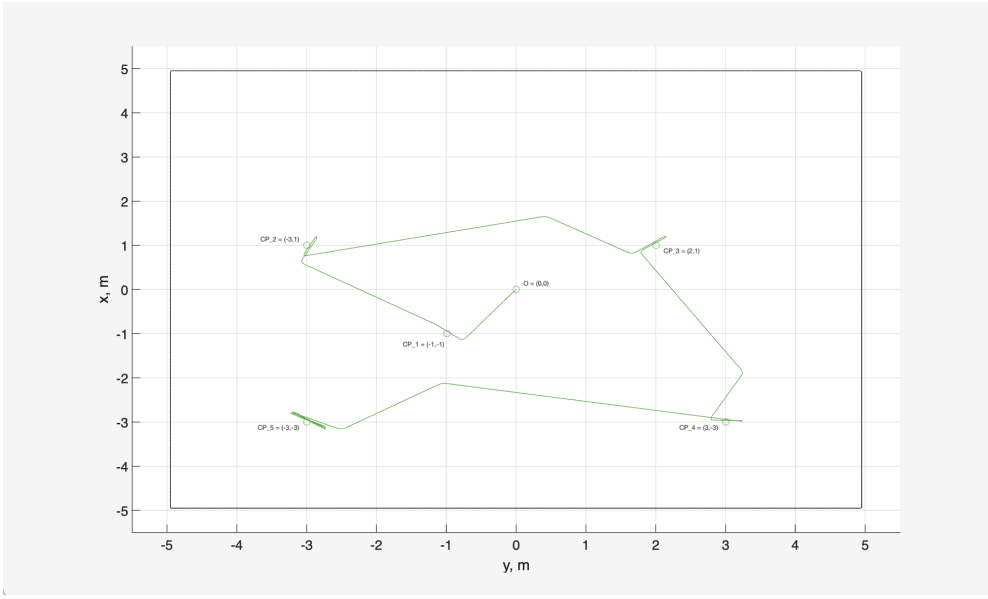

Figure 2: Task 1 trajectory in open space. The path shows that the waypoint controller reaches the target without obstacle handling, but does so with non-ideal alignment and curved approach segments rather than perfect straight-line convergence.

4.4 Task 2: obstacle avoidance and hybrid control

4.4.1 ANN obstacle avoidance

The obstacle avoidance layer was based on the neural-network controller developed in the lab exercises. It uses the two sensor inputs and a fixed set of weights:

$$N_1 = -0.5L_S + 1.0R_S, \qquad N_2 = 1.0L_S - 0.5R_S$$

Each neuron is compared against a threshold of 0.3. If the threshold is exceeded, the corresponding motor is commanded on; otherwise it is off. In the final tuned script, the ANN outputs were 0 V or 2 V.

This controller works when the obstacle is closer on one side than the other. However, it struggles with symmetry. If the robot faces a wall almost head-on, the two sensor readings become similar and the ANN can stop making directional decisions. Reviews of reactive obstacle avoidance and recovery report the same kind of failure: the robot can stall or oscillate without making progress around the obstacle [1, 3]. This was a reason for adding a recovery layer.

A fixed recovery routine was tested before the final wall-follow design. The idea was to trigger a short escape manoeuvre when the ANN stalled, then return control to the hybrid controller. In practice, this did not integrate well with waypoint tracking. The robot could resume forward motion while still facing a wall and clip the obstacle, or the forced recovery turn could conflict with the waypoint command when they demanded different directions. For that reason, the fixed recovery routine was dropped and replaced by a Bug-style wall-follow mode.

4.4.2 Wall-follow recovery

Wall-follow was added to restore progress when the ANN became ineffective. The mode is activated when obstacle avoidance is already active and the robot is either moving too slowly for several steps or is judged to be facing a wall directly. This follows the literature on boundary following and recovery: wall-follow is used here as a recovery behaviour rather than the default obstacle strategy [4, 3].

Once activated, the wall-follow side remains fixed for that wall-follow episode. The side-selection logic was made more robust than a simple comparison of left and right sensor distances. In frontal cases, where the two clearances are almost equal, the side is chosen from the waypoint heading error. In non-frontal cases, the controller compares left and right clearances using a small margin so that tiny sensor differences do not trigger the wrong side choice.

The wall-follow commands themselves were kept simple. The robot either turns strongly away from a head-on wall, turns slightly to move closer to or further from the wall, or drives forward when the wall distance is acceptable. This is not a geometric wall-tracking controller in the classical sense because the robot does not have real side sensors. It is a heuristic wall-follow behaviour built from forward sensor information. The aim was to add only the recovery logic needed for a repeatable failure mode.

4.4.3 Controller arbitration

The final controller uses a supervisory hierarchy:

$$\text{stop} > \text{wall-follow} > \text{ANN avoidance} > \text{waypoint tracking}$$

This means the robot stops once the final waypoint is reached. In the final implementation, this is handled with a latched mission-complete flag: when the robot first enters the tolerance region of the last checkpoint, the flag is set and both motor voltages remain at zero for the rest of the simulation. This avoids the earlier problem where the robot could briefly satisfy the stop condition, coast beyond the target because of inertia, and then restart. If the mission is not complete and wall-follow is active, wall-follow commands override the other layers. If wall-follow is inactive but obstacle avoidance is active, the ANN commands are applied. Only when no higher-priority mode is active does the waypoint controller drive the robot towards the next checkpoint.

This structure is part of the final design. The individual controllers are all basic, but the supervisory logic gives each one a role. Keeping the controller split into parts made it easier to verify each layer, debug failures, and justify the final design.

4.5 Parameter selection and tuning

Most parameters were selected by observation rather than formal optimisation. The controller was repeatedly simulated, failure cases were noted, and the thresholds were adjusted to remove specific problems without changing the overall architecture. This was a verification-led tuning process rather than blind trial and error: each parameter change was tied to an observed issue such as clipping, premature switching, or heading overshoot.

The tuning decisions were:

- reducing the original drive and turn voltages to make the robot less aggressive,
- inflating the body clearance margin to reduce visual clipping through walls,
- using hysteresis between avoidance entry and exit thresholds,
- adding a weak-turn band in the waypoint controller to reduce heading overshoot,
- refining wall-follow side selection for frontal wall cases.

The final values used in the main script are listed in Table 3. These values are not claimed to be globally optimal. They are the set that gave the best behaviour for the supplied environment and sensor model.

Table 3: Main tuning parameters used in the final hybrid controller.

Parameter	Value	Role
Simulation time	120 s	Allow full route to develop
Time step	0.05 s	Integration step
Robot clearance	0.22 m	Body inflation for sensing
Avoidance enter threshold	0.25 m	Enable avoidance
Avoidance exit threshold	0.35 m	Disable avoidance
Stuck speed threshold	0.05 m/s	Detect low progress
Side-selection margin	0.03 m	Robust wall side choice
Frontal-wall tolerance	0.05 m	Detect near-symmetric wall approach

5 Results

5.1 Task 1: waypoint tracking

The basic waypoint controller was able to reach checkpoints in open space using only the robot position and heading. This confirmed that the simulator interface was working correctly and that the controller could sequence between target points without obstacle handling, as illustrated by Figure 2 and Figure 3.

Its weakness was the heading transition. Because the robot rotates in place and then drives forward, any heading overshoot tends to appear as a zig-zag or curved approach to the waypoint rather than a straight line. The later weak-turn tuning reduced this behaviour, but did not remove it completely because the controller remains discrete and heuristic, again visible in Figure 2 and Figure 3.

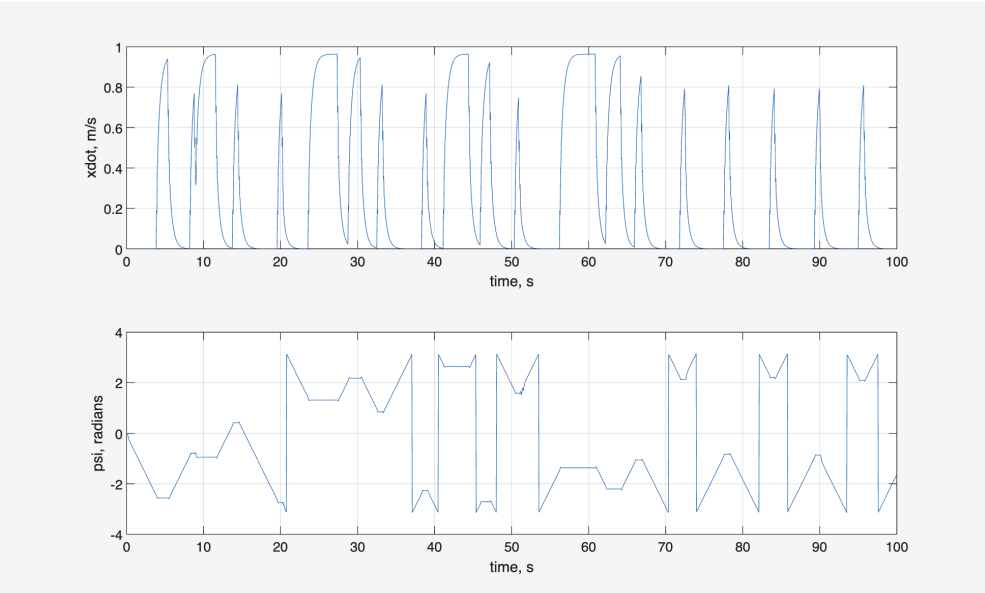

Figure 3: Task 1 forward speed and heading response. The state histories show the rotate-then-drive behaviour, with heading corrections followed by bursts of forward motion.

5.2 Task 2: hybrid waypoint tracking and obstacle avoidance

Task 2 used the final hybrid controller rather than a standalone obstacle-avoidance block. The ANN obstacle controller was one part of that structure. On its own, it produced the expected behaviour from the lab work: if an obstacle was more visible on one side, the robot tended to

steer away from it. This showed that a small network can produce local avoidance behaviour without a map.

The limitation was also evident. The ANN alone had no waypoint logic, no memory, and no recovery handling. In particular, frontal walls and near-symmetric situations produced indecisive behaviour. This matched the lab observations and was the reason for adding wall-follow and supervisory switching around it.

An intermediate fixed recovery routine was also tested, but it was dropped because it could clip walls after re-entering forward motion and could conflict with the waypoint layer when the forced turn direction disagreed with the checkpoint direction.

With the full hybrid controller enabled, the robot gave the best behaviour. The trajectory from the 120 s simulation shows that it was able to keep making progress around the internal obstacles instead of remaining trapped in front of them. The speed plot shows repeated bursts of forward motion separated by short turning or recovery phases, which is consistent with the intended behaviour of the supervisory controller, as shown in Figure 4, Figure 5, Figure 6, and Table 4.

The wall-follow mode was the improvement over the ANN-only version. When the robot approached a wall head-on or lost progress, wall-follow temporarily took over and guided the robot around the obstacle before returning control to the normal waypoint or avoidance layer. This did not produce a smooth or shortest path, but it did preserve mission progress. Figure 6 highlights three representative wall-follow segments.

The final path is still not ideal. There are sharp direction changes, some visible detours, and occasional switching transients. Even so, the final controller is much closer to a practical navigation solution than the pure waypoint or pure ANN controllers on their own, as summarised in Table 4 and Figures 4–5.

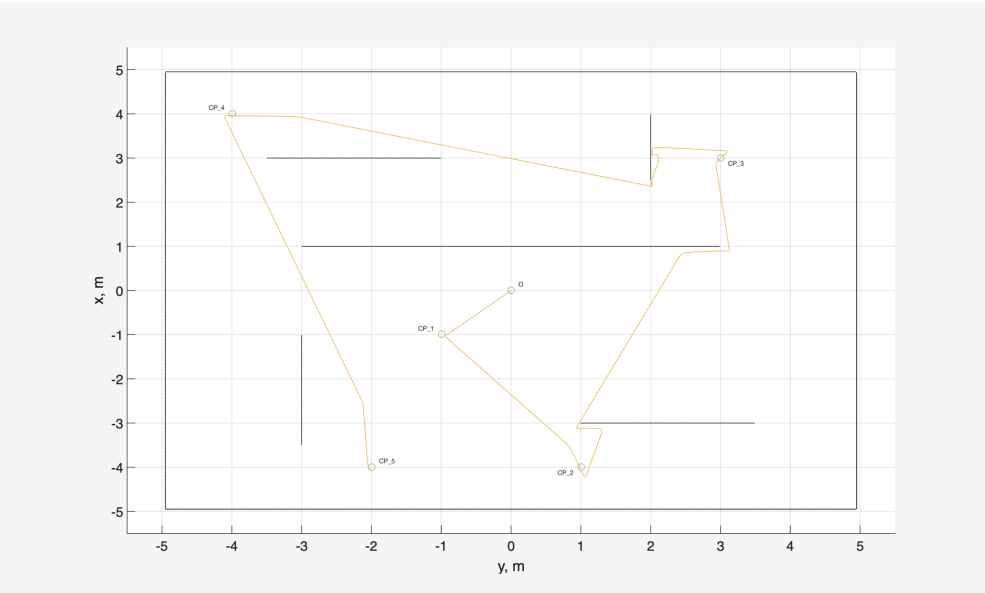

Figure 4: Final hybrid-controller trajectory in the supplied obstacle field. The deviations from straight segments correspond to avoidance and recovery actions.

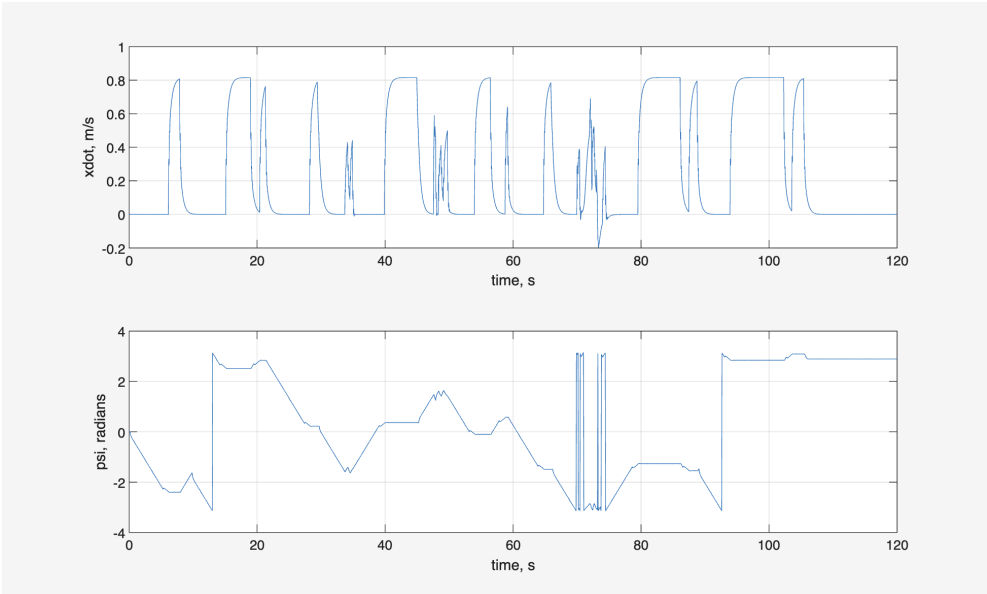

Figure 5: Forward speed and heading response for the final hybrid controller. The state histories show that progress is maintained through repeated transitions between waypoint tracking, obstacle avoidance, and recovery.

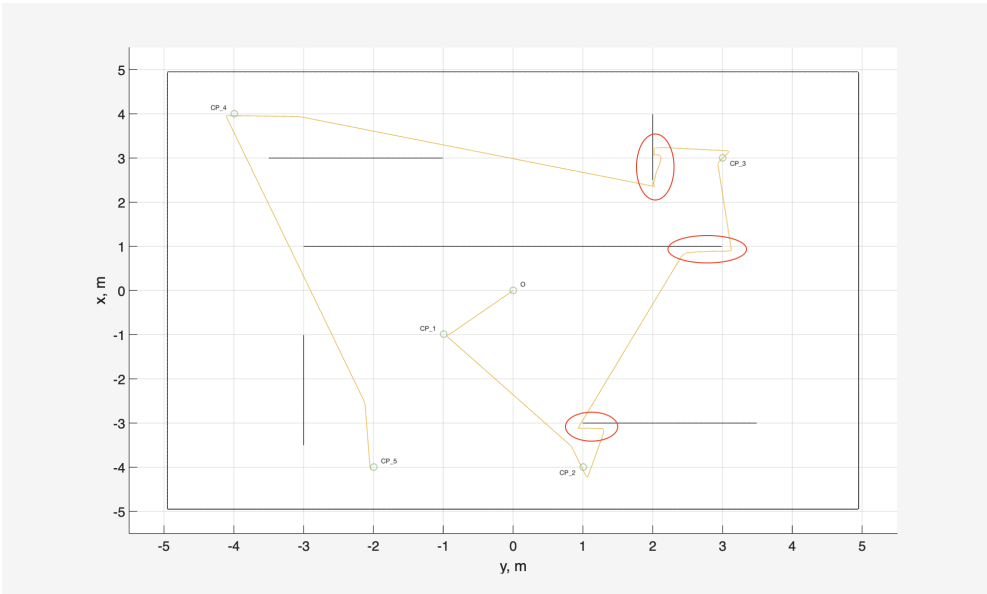

Figure 6: Representative wall-follow segments from the final controller. It shows the robot sliding along obstacle boundaries and then leaving them once forward progress towards the waypoint becomes possible again.

Table 4: Qualitative comparison of the main controller variants. The table shows why the final controller performs better overall: it is the only option that combines waypoint progress, obstacle response, and explicit recovery from deadlock.

Controller	Waypoint progress	Obstacle response	Recovery from deadlock
Waypoint only	Yes	No	No
ANN only	No	Yes	No
Altered ANN	No	Yes	Yes
Final hybrid	Yes	Yes	Yes

6 Discussion

6.1 Controller performance

The strength of the final system is the architecture rather than any single control law. The waypoint controller gives a goal direction. The ANN handles quick local reactions with almost no computational cost. The wall-follow mode deals with the specific situations where the ANN becomes ineffective. Together, these layers give a controller that is still small enough to analyse and verify, but more capable than any of the individual pieces in isolation.

The controller keeps the robot moving through the environment and allows it to negotiate obstacles that defeat the simpler lab controllers. It is less successful in terms of path efficiency. The route contains unnecessary turning, stop-start behaviour, and local detours. This is expected from a controller built around switching logic and very limited sensing, as seen in Figure 4 and Figure 5.

6.2 Limitations

The largest limitation is sensing. The robot only has two forward-looking distance measures. It does not know the full local geometry, and it does not have true side-wall sensing. As a result, several parts of the final controller are heuristic. Wall-follow side selection, wall-follow exit, and clearance tuning all depend on thresholds rather than geometry that is directly measured.

The waypoint controller is also limited. It is a rotate-then-drive controller, not a continuous path-following controller. That choice kept development and testing manageable, but it also means heading overshoot and zig-zag correction are built into the structure. The final weak-turn tuning reduced that problem, but did not remove it completely.

Finally, the controller is sensitive to tuning. Small changes in clearance thresholds, turning voltages, or switching margins can visibly change the trajectory. This is one of the costs of using a layered threshold-based design rather than a more model-based controller.

6.3 Comparison with literature

The final controller is closest in spirit to a hybrid local navigation method. The waypoint layer plays the role of goal-seeking. The wall-follow mode is similar to the boundary-following idea used in Bug-style methods: when direct progress to the goal is blocked, the robot temporarily follows the obstacle boundary until it is safe to leave [4].

The ANN layer is also consistent with the general idea of reactive local avoidance seen in VFH- and DWA-style systems, although it is simpler and does not explicitly model a local obstacle field or velocity search space [5, 6, 7]. In that sense, the final controller is not a state-of-the-art navigation method, but it follows the same broad lesson from the literature: local avoidance alone is not enough, and recovery logic matters.

6.4 Potential improvements

There are several straightforward ways to improve the current system. The first is better sensing. Even one additional forward sensor or a pair of true side sensors would make wall-follow far more reliable. The second is smoother heading control, so the robot can converge to a waypoint direction without repeated stop-turn-drive cycles. The third is better exit logic for wall-follow, possibly based on persistence or a small local memory rather than a single threshold.

Beyond that, the controller could be extended with a simple local map or cross-track error term so that it follows a path segment rather than repeatedly pointing at the next checkpoint. That would improve path quality, but it would also move the design away from the low-complexity layered controller used here. The next step should be to add complexity only where the current structure can no longer support the required performance.

7 Conclusion

This work developed a hybrid controller for the ENG5009 mobile robot simulator assignment using the supplied model and lab material as a starting point. A waypoint tracker was first used to solve the point-to-point task. A threshold ANN obstacle controller was then integrated for local obstacle response. Finally, a wall-follow recovery mode and a supervisory arbitration layer were added to overcome the ANN deadlock cases seen near walls.

The result is that a small set of simple controllers can achieve reliable navigation in the supplied environment when they are combined with switching logic. The final controller is not optimal, and it still shows the limits of narrow sensing and threshold tuning. Its value is that it follows a design principle: start with the simplest controller that can meet the task, verify its behaviour carefully, and add logic only when specific failure cases require it. On that basis, the final controller demonstrates the engineering ideas behind hybrid mobile robot control.

8 References

References

[1] Y. Wang, X. Li, J. Zhang, S. Li, Z. Xu, and X. Zhou, "Review of wheeled mobile robot collision avoidance under unknown environment, *Science Progress*, vol. 104, no. 3, 2021.

[2] L. Liu, X. Wang, X. Yang, H. Liu, J. Li, and P. Wang, "Path planning techniques for mobile robots: Review and prospect, *Expert Systems with Applications*, vol. 227, 2023.

[3] Y. Doi, K. Hosoi, T. Murayama, and Y. Uchimura, "Detection and Recovery Method for Immobilized Mobile Robot, *Journal of Robotics and Mechatronics*, vol. 37, no. 6, 2025.

[4] V. J. Lumelsky and A. A. Stepanov, "Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape, *Algorithmica*, vol. 2, 1987.

[5] J. Borenstein and Y. Koren, "The Vector Field Histogram: Fast Obstacle Avoidance for Mobile Robots, *IEEE Transactions on Robotics and Automation*, vol. 7, no. 3, 1991.

[6] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance, *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, 1997.

[7] I. Ulrich and J. Borenstein, "VFH+: reliable obstacle avoidance for fast mobile robots, in *Proc. IEEE International Conference on Robotics and Automation*, 1998.

[8] J.-F. Duhe, S. Victor, and P. Melchior, "Contributions on Artificial Potential Field Method for Effective Obstacle Avoidance, *Fractional Calculus and Applied Analysis*, vol. 24, no. 2, 2021.

9 Appendices

9.1 Lab 1 results

Lab 1 focused on the MATLAB Fuzzy Logic Toolbox. The outcome was not the final assignment controller, but familiarity with fuzzy sets, rule bases, and defuzzification. This introduced heuristic control design and rule-based reasoning before the robot simulator was used. The full laboratory answer sheet is appended at the end of this report.

9.2 Lab 2 results

Lab 2 provided the practical bridge into the robot simulator. The basic manoeuvre script confirmed that the model and plotting loop were working. The fuzzy controller showed how a purely reactive controller can become trapped in repeated local motion. The neural-network controller showed how a tiny threshold network can generate obstacle response, but also exposed the symmetry deadlock that later motivated the wall-follow recovery layer. The full laboratory answer sheet is appended at the end of this report.

Table 5 shows how each lab script contributed a specific lesson to the final controller design.

Table 5: Summary of the main lessons from the lab scripts.

Script or exercise	Main lesson
Basic manoeuvre	Verified simulator integration and open-loop robot response
Fuzzy controller	Reactive control can avoid some obstacles, but can also oscillate or stall without mission logic
Neural controller	Very simple threshold logic can steer the robot, but frontal symmetry causes deadlock
Altered neural controller	Explicit recovery improves robustness when the basic ANN stops making progress